

UART Transmitter (UART_TX) RTL Design and Verification Documentation

Digital Design Verification Report by: Yassin Kamel

August 27, 2026

Contents

1	Introduction	2
2	RTL Implementation	2
2.1	FSM Controller	2
2.2	Parity Calculator	3
2.3	Serialiser	4
2.4	Multiplexer	4
2.5	Top-Level Integration	5
3	Testbench Verification	6
4	Simulation Results	8
4.1	Test 1	8
4.2	Test 2	8
4.3	Test 3	9
4.4	Test 4	9
4.5	Test 5	9
4.6	Logs	10
5	Timing Analysis Reports	11
5.1	Setup Summary 1	11
5.2	Hold Summary 1	12
5.3	Setup Summary 2	13
5.4	Hold Summary 2	14
5.5	Setup Summary 3	15
5.6	Hold Summary 3	16
5.7	Setup Summary 4	17
5.8	Hold Summary 5	18

1 Introduction

This document is a thorough documentation for the UART TX module project. The module is composed of an FSM controller, a parity calculator, a serialiser and a multiplexer which handles the output of these modules based on the controller.

2 RTL Implementation

2.1 FSM Controller

The controller manages the transmission states (free, start, data, parity, and stop) which represent which state the data transmission has reached. Using these states, the module is able to control the rest of the modules properly and in time.

```
1 module controller(  
2     input wire  clk, v_input, p_en, rst, data_done,  
3     output reg [1:0] mux_sel,  
4     output reg  shift_en, busy, load  
5 );  
6  
7     localparam free = 3'b000, start = 3'b001, data = 3'b010, parity = 3'  
8         b011, stop = 3'b100;  
9     reg [2:0] state;  
10  
11     always @(posedge clk or negedge rst) begin  
12         if(!rst) begin  
13             state <= free;  
14             busy <= 0;  
15             shift_en <= 0;  
16             load <= 0;  
17             mux_sel <= 2'b11;  
18         end  
19         else begin  
20             if(state == free && v_input) begin  
21                 state <= start;  
22                 mux_sel <= 2'b00;  
23                 busy <= 1;  
24                 shift_en <= 0;  
25                 load <= 1;  
26             end  
27             else if(state == start) begin  
28                 state <= data;  
29                 mux_sel <= 2'b01;  
30                 shift_en <= 1;  
31                 busy <= 1;  
32                 load <= 0;  
33             end  
34             else if(state == data && !data_done) begin  
35                 state <= data;  
36                 mux_sel <= 2'b01;  
37                 shift_en <= 1;  
38                 busy <= 1;  
39                 load <= 0;  
40             end  
41             else if(state == data && data_done && p_en) begin  
42                 state <= parity;  
43             end  
44         end  
45     end
```

```

42         mux_sel <= 2'b10;
43         shift_en <= 0;
44         busy <= 1;
45         load <= 0;
46     end
47     else if(state == data && data_done && !p_en) begin
48         state <= stop;
49         mux_sel <= 2'b11;
50         shift_en <= 0;
51         busy <= 1;
52         load <= 0;
53     end
54     else if(state == parity) begin
55         state <= stop;
56         mux_sel <= 2'b11;
57         shift_en <= 0;
58         busy <= 1;
59         load <= 0;
60     end
61     else if(state == stop) begin
62         state <= free;
63         mux_sel <= 2'b11;
64         shift_en <= 0;
65         busy <= 0;
66         load <= 0;
67     end
68 end
69 end
70 endmodule

```

Listing 1: FSM Controller Module

2.2 Parity Calculator

This module calculates the parity bit real-time by XOR-ing the entire data array and the parity type given to the machine. Verilog should be able to efficiently synthesize this logical calculation better than a for loop over the array.

```

1 module parity_calc #(parameter data_width = 8)(
2     input wire [data_width-1:0] p_data,
3     input wire p_bit,
4     output wire parity_out
5 );
6     assign parity_out = (^p_data) ^ p_bit;
7 endmodule

```

Listing 2: Parity Calculator Module

2.3 Serialiser

This module receives the parallel input and stores it into a shift-register, shifts the data with each clock cycle while wiring the LSB of the shift-register to the output and marks the done flag based on the counted shifted bits.

```
1 module parallel_to_serial #(parameter data_width = 8)(
2     input wire clk,rst,load,shift_en,
3     input wire [data_width-1:0] parallel_in,
4     output wire serial_out,data_done
5 );
6
7     reg [data_width-1:0] shift_reg;
8     reg [$clog2(data_width)-1:0] count = 0;
9
10    assign serial_out = shift_reg[0];
11    assign data_done = (count==data_width-1);
12
13    always @(posedge clk or negedge rst) begin
14        if (~rst) begin
15            shift_reg <= 0;
16            count <= 0;
17        end
18        else if (load) begin
19            shift_reg <= parallel_in;
20            count <= 0;
21        end
22        else if (shift_en) begin
23            shift_reg <= {1'b0,shift_reg[data_width-1:1]};
24            if (count != data_width - 1) begin
25                count <= count + 1'b1;
26            end
27        end
28        else begin
29            count <= 0;
30        end
31    end
32
33 endmodule
```

Listing 3: Serialiser Module

2.4 Multiplexer

This is a simple standard 4-input multiplexer. The trick lies in its usage which will be explained in the Top Module section.

```
1 module mux4x2(
2     input wire [3:0] data,
3     input wire [1:0] mux_sel,
4     output wire mux_out
5 );
6     assign mux_out = data[mux_sel];
7 endmodule
```

Listing 4: Multiplexer Module

2.5 Top-Level Integration

This module wires all the sub-modules together with multiple internal wires declared as shown in the code. The multiplexer is wired to the following inputs:

- 1 which identifies the idle free state of the transmitter AND the stop bit.
- The calculated parity input which is outputted in the parity state.
- The serialiser's output which is outputted in the data state.
- 0 which identifies the start bit.

So due to the overlap between the idle and stop state outputs, the MUX is able to reduce the 5 states into 4 possible outputs.

```
1 module UART_TX#(parameter data_width = 8)(
2     input wire clk,rst,p_bit,p_en,v_input,
3     input wire [data_width-1:0] p_input,
4     output wire TX_output,busy
5 );
6
7     wire data_done_in,shift_en_in,load_in,parity_out_in,serial_out_in;
8     wire [1:0] mux_sel_in;
9
10    controller Controller(
11        .clk(clk),
12        .rst(rst),
13        .v_input(v_input),
14        .p_en(p_en),
15        .data_done(data_done_in),
16        .mux_sel(mux_sel_in),
17        .shift_en(shift_en_in),
18        .busy(busy),
19        .load(load_in)
20    );
21
22    parity_calc #(.data_width(data_width)) Parity(
23        .p_data(p_input),
24        .p_bit(p_bit),
25        .parity_out(parity_out_in)
26    );
27
28    parallel_to_serial #(.data_width(data_width)) Serialiser(
29        .clk(clk),
30        .rst(rst),
31        .load(load_in),
32        .shift_en(shift_en_in),
33        .parallel_in(p_input),
34        .serial_out(serial_out_in),
35        .data_done(data_done_in)
36    );
37
38    mux4x2 Mux(
39        .data({1'b1,parity_out_in,serial_out_in,1'b0}),
40        .mux_sel(mux_sel_in),
41        .mux_out(TX_output)
42    );
43 endmodule
```

Listing 5: Top-Level UART_TX Module

3 Testbench Verification

This testbench is made to test the top-level module and not the individual modules. It has been split into the reset task to clear everything and the `send_data`. The `send_data` task waits for a clock cycle, sets the inputs and raises the `v_input` signal to let the module load the data and then it waits for another clock cycle for this data to be processed and lowers the `v_input` signal. A wait statement was necessary due to the minuscule delay that the busy signal takes to flip on the clock signal then it uses a while statement to display the updated output of the module with each clock cycle (a 1 ns delay is added to make sure the output is updated). The tests that were chosen:

- Test 1: 10101010 with even parity, which means the parity bit should be 0.
- Test 2: 11101010 with even parity, which means the parity bit should be 1.
- Test 3: 11001100 with odd parity, which means the parity bit should be 1.
- Test 4: 11101100 with odd parity, which means the parity bit should be 0.
- Test 5: 11110000 with disabled parity, which means the parity bit should not be outputted at all.

```
1  `timescale 1ns / 1ps
2
3  module UART_TX_TB;
4
5      parameter data_width = 8, clk_period = 20;
6      reg clk, rst, p_bit, p_en, v_input;
7      reg [data_width-1:0] p_input;
8      wire TX_output, busy;
9
10     UART_TX #(data_width(data_width)) DUT (
11         .clk(clk),
12         .rst(rst),
13         .p_bit(p_bit),
14         .p_en(p_en),
15         .v_input(v_input),
16         .p_input(p_input),
17         .TX_output(TX_output),
18         .busy(busy)
19     );
20
21     always #(clk_period/2) begin
22         clk = ~clk;
23     end
24
25     task reset();
26     begin
27         rst = 0;
28         v_input = 0;
29         p_en = 0;
30         p_bit = 0;
31         p_input = 0;
32         #(clk_period * 10);
33         rst = 1;
34         #(clk_period * 2);
```

```

35     end
36 endtask
37
38 task send_data(input [data_width-1:0] data,input parity_enable,
39               input parity_type);
40 begin
41     @(posedge clk);
42     p_input = data;
43     p_en = parity_enable;
44     p_bit = parity_type;
45     v_input = 1;
46     @(posedge clk);
47     v_input = 0;
48     wait(busy == 1);
49     while (busy == 1) begin
50         #1;
51         if(busy == 1)
52             $display("Time: %0t          Serial output: %b", $time,
53                     TX_output);
54         @(posedge clk);
55     end
56     #(clk_period * 5);
57 end
58 endtask
59
60 initial begin
61     clk = 0;
62     reset();
63     $display("Time: %0t          Data: 10101010          Even
64             Parity", $time);
65     send_data(8'b10101010,1'b1,1'b0);
66     $display("Time: %0t          Data: 10101010          Even
67             Parity", $time);
68     send_data(8'b11101010,1'b1,1'b0);
69     $display("Time: %0t          Data: 11001100          Odd Parity
70             ", $time);
71     send_data(8'b11001100,1'b1,1'b1);
72     $display("Time: %0t          Data: 11001100          Odd Parity
73             ", $time);
74     send_data(8'b11101100,1'b1,1'b1);
75     $display("Time: %0t          Data: 11110000          No Parity"
76             , $time);
77     send_data(8'b11110000,1'b0,1'b0);
78     $finish;
79 end
80 endmodule

```

Listing 6: UART Transmitter Testbench

4 Simulation Results

The figures below contain the waveform and the logs of the testbench code when simulated.

4.1 Test 1

The regions of this test have been labeled for further explanation and understanding, same labeling should apply to the rest of the waveforms.

- Region 0: This region is the idle free region with an active high output.
- Region 1: This region is the region which the `v_input` is activated to load the input data.
- Region 2: This region is the start bit which is shown to be low in the serial output stream.
- Regions 3-10: This region is the data bits indexed from 0 to 7.
- Region 11: This region is the parity bit which is low in this test.
- Region 12: This region is the stop bit which is high.
- Region 13: This region is the idle free region reached once more where the busy flag became low again.

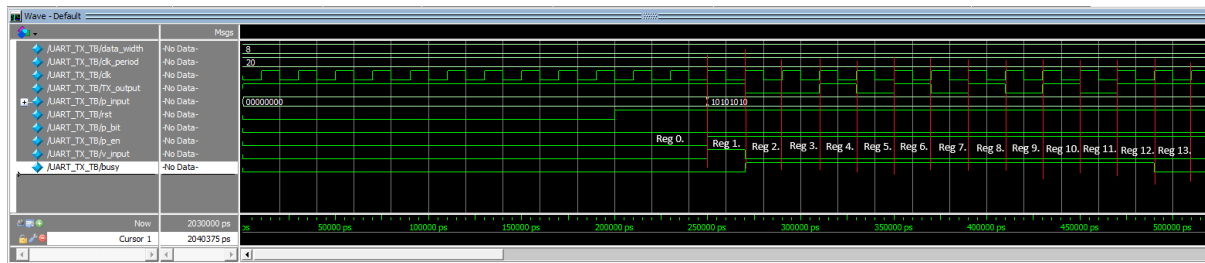


Figure 1: Modelsim Waveform for Test 1

4.2 Test 2

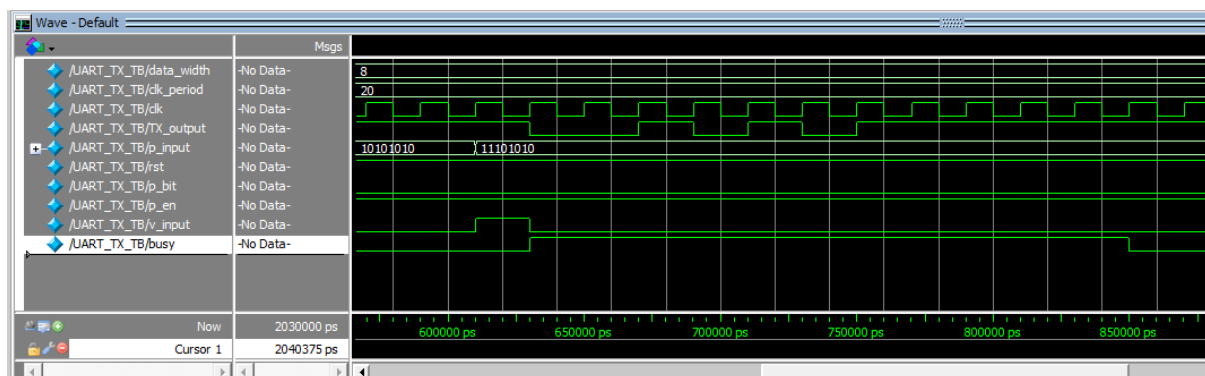


Figure 2: Modelsim Waveform for Test 2

4.3 Test 3

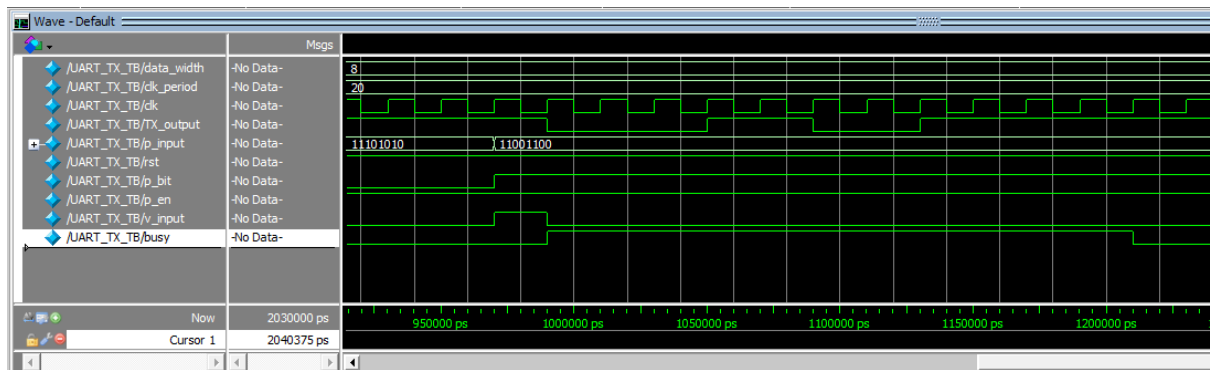


Figure 3: Modelsim Waveform for Test 3

4.4 Test 4

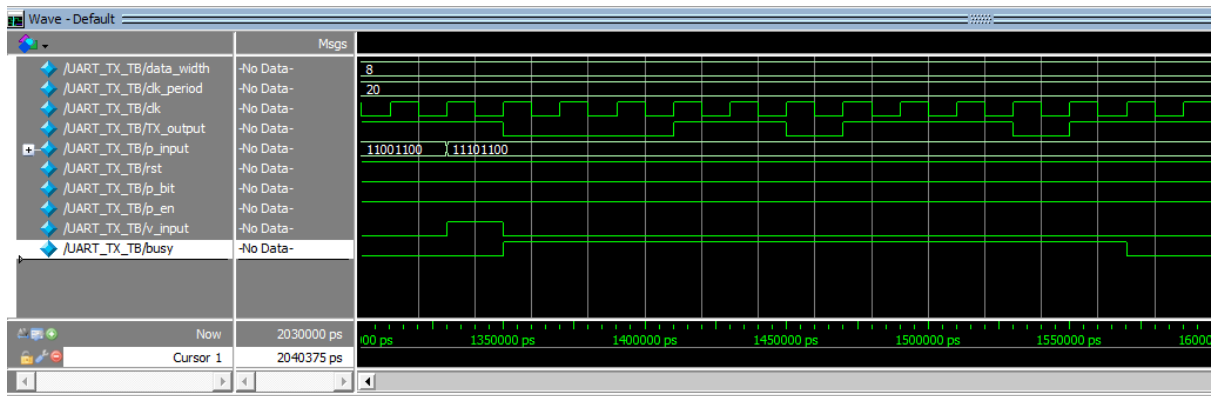


Figure 4: Modelsim Waveform for Test 4

4.5 Test 5

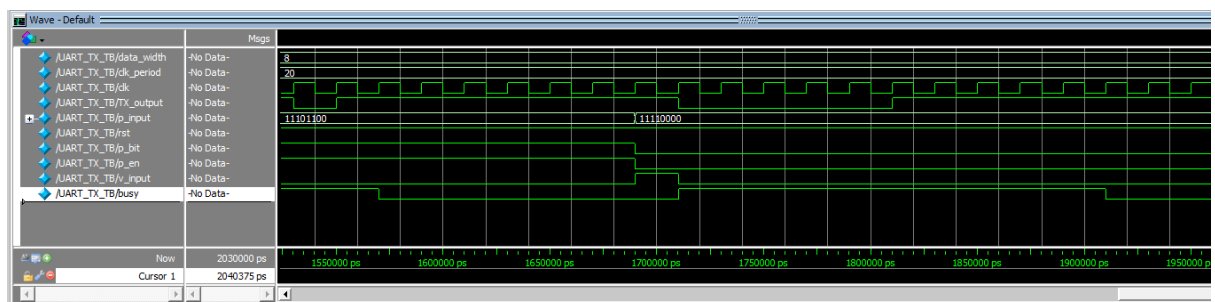


Figure 5: Modelsim Waveform for Test 5

4.6 Logs

```
VSIM 25> Run -all
# Time: 240000      Data: 10101010      Even Parity
# Time: 271000      Serial output: 0
# Time: 291000      Serial output: 0
# Time: 311000      Serial output: 1
# Time: 331000      Serial output: 0
# Time: 351000      Serial output: 1
# Time: 371000      Serial output: 0
# Time: 391000      Serial output: 1
# Time: 411000      Serial output: 0
# Time: 431000      Serial output: 1
# Time: 451000      Serial output: 0
# Time: 471000      Serial output: 1
# Time: 610000      Data: 10101010      Even Parity
# Time: 631000      Serial output: 0
# Time: 651000      Serial output: 0
# Time: 671000      Serial output: 1
# Time: 691000      Serial output: 0
# Time: 711000      Serial output: 1
# Time: 731000      Serial output: 0
# Time: 751000      Serial output: 1
# Time: 771000      Serial output: 1
# Time: 791000      Serial output: 1
# Time: 811000      Serial output: 1
# Time: 831000      Serial output: 1
# Time: 970000      Data: 11001100      Odd Parity
# Time: 991000      Serial output: 0
# Time: 1011000     Serial output: 0
# Time: 1031000     Serial output: 0
# Time: 1051000     Serial output: 1
# Time: 1071000     Serial output: 1
# Time: 1091000     Serial output: 0
# Time: 1111000     Serial output: 0
# Time: 1131000     Serial output: 1
# Time: 1151000     Serial output: 1
# Time: 1171000     Serial output: 1
# Time: 1191000     Serial output: 1
# Time: 1330000     Data: 11001100      Odd Parity
# Time: 1351000     Serial output: 0
# Time: 1371000     Serial output: 0
# Time: 1391000     Serial output: 0
# Time: 1411000     Serial output: 1
# Time: 1431000     Serial output: 1
# Time: 1451000     Serial output: 0
# Time: 1471000     Serial output: 1
# Time: 1491000     Serial output: 1
# Time: 1511000     Serial output: 1
# Time: 1531000     Serial output: 0
# Time: 1551000     Serial output: 1
# Time: 1690000     Data: 11110000      No Parity
# Time: 1711000     Serial output: 0
# Time: 1731000     Serial output: 0
# Time: 1751000     Serial output: 0
# Time: 1771000     Serial output: 0
# Time: 1791000     Serial output: 0
# Time: 1811000     Serial output: 1
# Time: 1831000     Serial output: 1
# Time: 1851000     Serial output: 1
# Time: 1871000     Serial output: 1
# Time: 1891000     Serial output: 1
# ** Note: $finish      : D:/desktop/NTI/FPGA/Verilog Final Project UART/UART_TX_TB.v(70)
```

Figure 6: Modelsim Logs for all the previous tests by the testbench

5 Timing Analysis Reports

5.1 Setup Summary 1

Setup Summary report for UART_TX

Thu Aug 27 19:12:35 2026

Quartus Prime Version 25.1std.0 Build 1129 10/21/2025 SC Lite Edition

; Table of Contents ;

1. Legal Notice
2. Slow 1100mV 85C Model Setup Summary

; Legal Notice ;

Copyright (C) 2025 Altera Corporation. All rights reserved.
Your use of Altera Corporation's design tools, logic functions
and other software and tools, and any partner logic
functions, and any output files from any of the foregoing
(including device programming or simulation files), and any
associated documentation or information are expressly subject
to the terms and conditions of the Altera Program License
Subscription Agreement, the Altera Quartus Prime License Agreement,
the Altera IP License Agreement, or other applicable license
agreement, including, without limitation, that your use is for
the sole purpose of programming logic devices manufactured by
Altera and sold by Altera or its authorized distributors. Please
refer to the Altera Software License Subscription Agreements
on the Quartus Prime software download page.

+-----+
; Slow 1100mV 85C Model Setup Summary ;
+-----+
; Clock ; Slack ; End Point TNS ;
+-----+
; clk ; 17.982 ; 0.000 ;
+-----+

5.2 Hold Summary 1

Hold Summary report for UART_TX

Thu Aug 27 19:12:37 2026

Quartus Prime Version 25.1std.0 Build 1129 10/21/2025 SC Lite Edition

; Table of Contents ;

1. Legal Notice
2. Slow 1100mV 85C Model Hold Summary

; Legal Notice ;

Copyright (C) 2025 Altera Corporation. All rights reserved.
Your use of Altera Corporation's design tools, logic functions
and other software and tools, and any partner logic
functions, and any output files from any of the foregoing
(including device programming or simulation files), and any
associated documentation or information are expressly subject
to the terms and conditions of the Altera Program License
Subscription Agreement, the Altera Quartus Prime License Agreement,
the Altera IP License Agreement, or other applicable license
agreement, including, without limitation, that your use is for
the sole purpose of programming logic devices manufactured by
Altera and sold by Altera or its authorized distributors. Please
refer to the Altera Software License Subscription Agreements
on the Quartus Prime software download page.

+-----+
; Slow 1100mV 85C Model Hold Summary ;
+-----+
; Clock ; Slack ; End Point TNS ;
+-----+
; clk ; 0.290 ; 0.000 ;
+-----+

5.3 Setup Summary 2

Setup Summary report for UART_TX

Thu Aug 27 19:12:47 2026

Quartus Prime Version 25.1std.0 Build 1129 10/21/2025 SC Lite Edition

; Table of Contents ;

1. Legal Notice
2. Slow 1100mV OC Model Setup Summary

; Legal Notice ;

Copyright (C) 2025 Altera Corporation. All rights reserved.
Your use of Altera Corporation's design tools, logic functions
and other software and tools, and any partner logic
functions, and any output files from any of the foregoing
(including device programming or simulation files), and any
associated documentation or information are expressly subject
to the terms and conditions of the Altera Program License
Subscription Agreement, the Altera Quartus Prime License Agreement,
the Altera IP License Agreement, or other applicable license
agreement, including, without limitation, that your use is for
the sole purpose of programming logic devices manufactured by
Altera and sold by Altera or its authorized distributors. Please
refer to the Altera Software License Subscription Agreements
on the Quartus Prime software download page.

+-----+
; Slow 1100mV OC Model Setup Summary ;
+-----+
; Clock ; Slack ; End Point TNS ;
+-----+
; clk ; 18.031 ; 0.000 ;
+-----+

5.4 Hold Summary 2

Hold Summary report for UART_TX

Thu Aug 27 19:12:50 2026

Quartus Prime Version 25.1std.0 Build 1129 10/21/2025 SC Lite Edition

; Table of Contents ;

1. Legal Notice
2. Slow 1100mV OC Model Hold Summary

; Legal Notice ;

Copyright (C) 2025 Altera Corporation. All rights reserved.
Your use of Altera Corporation's design tools, logic functions
and other software and tools, and any partner logic
functions, and any output files from any of the foregoing
(including device programming or simulation files), and any
associated documentation or information are expressly subject
to the terms and conditions of the Altera Program License
Subscription Agreement, the Altera Quartus Prime License Agreement,
the Altera IP License Agreement, or other applicable license
agreement, including, without limitation, that your use is for
the sole purpose of programming logic devices manufactured by
Altera and sold by Altera or its authorized distributors. Please
refer to the Altera Software License Subscription Agreements
on the Quartus Prime software download page.

+-----+
; Slow 1100mV OC Model Hold Summary ;
+-----+-----+-----+
; Clock ; Slack ; End Point TNS ;
+-----+-----+-----+
; clk ; 0.259 ; 0.000 ;
+-----+-----+-----+

5.5 Setup Summary 3

Setup Summary report for UART_TX

Thu Aug 27 19:12:57 2026

Quartus Prime Version 25.1std.0 Build 1129 10/21/2025 SC Lite Edition

; Table of Contents ;

1. Legal Notice
2. Fast 1100mV 85C Model Setup Summary

; Legal Notice ;

Copyright (C) 2025 Altera Corporation. All rights reserved.
Your use of Altera Corporation's design tools, logic functions
and other software and tools, and any partner logic
functions, and any output files from any of the foregoing
(including device programming or simulation files), and any
associated documentation or information are expressly subject
to the terms and conditions of the Altera Program License
Subscription Agreement, the Altera Quartus Prime License Agreement,
the Altera IP License Agreement, or other applicable license
agreement, including, without limitation, that your use is for
the sole purpose of programming logic devices manufactured by
Altera and sold by Altera or its authorized distributors. Please
refer to the Altera Software License Subscription Agreements
on the Quartus Prime software download page.

+-----+
; Fast 1100mV 85C Model Setup Summary ;
+-----+
; Clock ; Slack ; End Point TNS ;
+-----+
; clk ; 18.874 ; 0.000 ;
+-----+

5.6 Hold Summary 3

Hold Summary report for UART_TX

Thu Aug 27 19:13:00 2026

Quartus Prime Version 25.1std.0 Build 1129 10/21/2025 SC Lite Edition

; Table of Contents ;

1. Legal Notice
2. Fast 1100mV 85C Model Hold Summary

; Legal Notice ;

Copyright (C) 2025 Altera Corporation. All rights reserved.
Your use of Altera Corporation's design tools, logic functions
and other software and tools, and any partner logic
functions, and any output files from any of the foregoing
(including device programming or simulation files), and any
associated documentation or information are expressly subject
to the terms and conditions of the Altera Program License
Subscription Agreement, the Altera Quartus Prime License Agreement,
the Altera IP License Agreement, or other applicable license
agreement, including, without limitation, that your use is for
the sole purpose of programming logic devices manufactured by
Altera and sold by Altera or its authorized distributors. Please
refer to the Altera Software License Subscription Agreements
on the Quartus Prime software download page.

+-----+
; Fast 1100mV 85C Model Hold Summary ;
+-----+
; Clock ; Slack ; End Point TNS ;
+-----+
; clk ; 0.174 ; 0.000 ;
+-----+

5.7 Setup Summary 4

Setup Summary report for UART_TX

Thu Aug 27 19:13:06 2026

Quartus Prime Version 25.1std.0 Build 1129 10/21/2025 SC Lite Edition

; Table of Contents ;

1. Legal Notice
2. Fast 1100mV OC Model Setup Summary

; Legal Notice ;

Copyright (C) 2025 Altera Corporation. All rights reserved.
Your use of Altera Corporation's design tools, logic functions
and other software and tools, and any partner logic
functions, and any output files from any of the foregoing
(including device programming or simulation files), and any
associated documentation or information are expressly subject
to the terms and conditions of the Altera Program License
Subscription Agreement, the Altera Quartus Prime License Agreement,
the Altera IP License Agreement, or other applicable license
agreement, including, without limitation, that your use is for
the sole purpose of programming logic devices manufactured by
Altera and sold by Altera or its authorized distributors. Please
refer to the Altera Software License Subscription Agreements
on the Quartus Prime software download page.

+-----+
; Fast 1100mV OC Model Setup Summary ;
+-----+
; Clock ; Slack ; End Point TNS ;
+-----+
; clk ; 18.937 ; 0.000 ;
+-----+

5.8 Hold Summary 5

Hold Summary report for UART_TX

Thu Aug 27 19:13:11 2026

Quartus Prime Version 25.1std.0 Build 1129 10/21/2025 SC Lite Edition

; Table of Contents ;

1. Legal Notice
2. Fast 1100mV OC Model Hold Summary

; Legal Notice ;

Copyright (C) 2025 Altera Corporation. All rights reserved.
Your use of Altera Corporation's design tools, logic functions
and other software and tools, and any partner logic
functions, and any output files from any of the foregoing
(including device programming or simulation files), and any
associated documentation or information are expressly subject
to the terms and conditions of the Altera Program License
Subscription Agreement, the Altera Quartus Prime License Agreement,
the Altera IP License Agreement, or other applicable license
agreement, including, without limitation, that your use is for
the sole purpose of programming logic devices manufactured by
Altera and sold by Altera or its authorized distributors. Please
refer to the Altera Software License Subscription Agreements
on the Quartus Prime software download page.

+-----+
; Fast 1100mV OC Model Hold Summary ;
+-----+
; Clock ; Slack ; End Point TNS ;
+-----+
; clk ; 0.156 ; 0.000 ;
+-----+